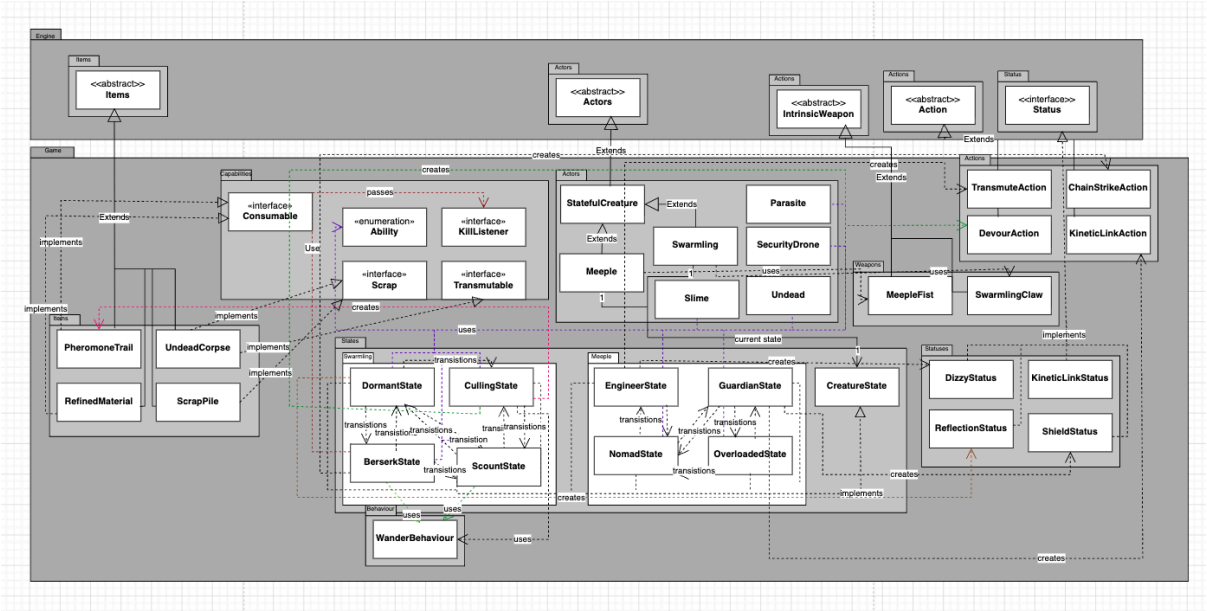
Design 1 State pattern and StatefulCreature
StatefulCreature extends Actor and encapsulates the common state machine logic including currentState, initialEntryDone, DizzyStatus checking, and the state transition flow. It also provides skipTurn() and passive() hook methods. Meeple and Swarmling extend StatefulCreature and only override their own specific logic.

pros	cons
Following the DRY principle, the state transition logic is implemented only once in `StatefulCreature`.	Added a layer of abstraction, making the class structure deeper
To implement OCP, simply inherit from `StatefulCreature` when adding a new creature.	
Satisfy SRP, each state class is responsible only for its own behavior and transition logic	

Design 2 Ability Enum Check
The state class uses `hasAbility()` to check the actor type. For Undead, it calls `enableAbility(Ability.IS_UNDEAD)`, and for Parasites, it calls `enableAbility(Ability.IS_PARASITE)`. The state class can identify a Parasite simply by calling `actor.hasAbility(Ability.IS_PARASITE)`.

pros	cons
Satisfy DIP, state classes do not depend on any specific Actor	need to manually call `enableAbility` in each Actor's constructor.
Satisfy OCP,To add a new parasite, simply call `enableAbility(IS_PARASITE)` without modifying any state classes.	
State classes do not need to import specific Actor classes.	
New entities are automatically detected by all statuses	



In our implementation, we chose design 1 and design 2. The introduction of `StatefulCreature` eliminated code repetition between `Meeple` and `Swarmling`. Each state class manages its own transition logic via `checkTransitions()`, satisfying the requirements of REQ5 Rule 2 (The next state must be deterministic), Rule 4 (The condition to transition between the states must NOT be trivial), and Rule 5 (The state transition itself must have an immediate effect). Furthermore, the use of the `Ability` enumeration ensures that all state classes are completely independent of specific Actor classes. `DormantState` checks for parasites using `hasAbility(IS_PARASITE)`, and `EngineerState` checks for undead using `hasAbility(IS_UNDEAD)`, complying with the DIP principle. If new parasite or undead types are added in the future, they can be automatically recognized by all states simply by calling `enableAbility()`, without requiring any modifications to the state classes, thereby satisfying the OCP principle.